```
# 0/1 Knapsack using Dynamic Programming
n = int(input("Enter number of items: "))
weights = []
profits = []
for i in range(n):
    w = int(input("Enter weight: "))
    p = int(input("Enter profit: "))

    weights.append(w)
    profits.append(p)
capacity = int(input("Enter knapsack capacity: "))
# Create DP table
dp = [[0 for j in range(capacity + 1)] for i in range(n + 1)]
# Fill DP table
for i in range(1, n + 1):
    for w in range(1, capacity + 1):
        if weights[i - 1] <= w:
            include = profits[i - 1] + dp[i - 1][w - weights[i - 1]]
            exclude = dp[i - 1][w]
            dp[i][w] = max(include, exclude)
        else:
            dp[i][w] = dp[i - 1][w]
# --- 1. Print the Formatted DP Table ---
print("\n--- DP Table ---")
# Print Column Headers (Capacities)
print(f"{'Item/Cap':<10}", " ".join(f"{c:3d}" for c in range(capacity + 1)))
# Print Row Values
for i in range(n + 1):
    row_label = f"Item {i}" if i > 0 else "0"
    row_values = " ".join(f"{dp[i][c]:3d}" for c in range(capacity + 1))
    print(f"{row_label:<10}{row_values}")
```

```
# --- 2. Backtracking to Find Selected Items ---
selected_items = []
w = capacity
for i in range(n, 0, -1):
    # Check if this item's profit was included to yield the current cell value
    if w >= weights[i - 1] and dp[i][w] == profits[i - 1] + dp[i - 1][w - weights[i - 1]]:
        selected_items.append(f"Item {i} (Weight: {weights[i - 1]}, Profit: {profits[i - 1]})")
        w -= weights[i - 1]
# Reverse the list to display selected items from first to last
selected_items.reverse()
# --- 3. Print the Final Summary Results ---
print("\n--- Summary Results ---")
print("Maximum Profit:", dp[n][capacity])
if selected_items:
    print("Selected Items:")
    for item in selected_items:
        print(f" - {item}")
else:
    print("Selected Items: None")
#if the table given
def solve_knapsack_alternative():
    items = ["A", "B", "C", "D"]
    weights = [8, 3, 2, 10]
    profits = [40, 30, 20, 50]
    capacity = 12
    n = len(items)

    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    # Populate the DP table
```

```
for i in range(1, n + 1):
    w_item = weights[i - 1]
    p_item = profits[i - 1]
    for w in range(capacity + 1):
        if w_item <= w:
            dp[i][w] = max(dp[i - 1][w], p_item + dp[i - 1][w - w_item])
        else:
            dp[i][w] = dp[i - 1][w]

# 1. Print the formatted DP table rows and columns
print("Item/Cap", " ".join(f"{c:3d}" for c in range(capacity + 1)))
row_labels = ["0"] + items
for i in range(n + 1):
    row_values = " ".join(f"{dp[i][c]:3d}" for c in range(capacity + 1))
    print(f"{row_labels[i]:<8}{row_values}")

# 2. Backtracking to favor the alternative tie-breaker path
selected_items = []
w = capacity
for i in range(n, 0, -1):
    w_item = weights[i - 1]
    p_item = profits[i - 1]
    if w >= w_item and dp[i][w] == p_item + dp[i - 1][w - w_item]:
        selected_items.append(items[i - 1])
        w -= w_item

selected_items.reverse()
# 3. Print maximum profit and selected items
print(f"\nMaximum Profit Obtained: {dp[n][capacity]}")
print(f"Selected Items: {', '.join(selected_items)}")
```

```
if __name__ == "__main__":
    solve_knapsack_alternative()
```

The 0/1 Knapsack Problem is an optimization problem in which we are given:A bag (knapsack) with a fixed capacity.A set of items, each having a weight and profit.We need to select items to get the maximum total profit without exceeding the bag's capacity.

Why is it called 0/1 Knapsack?1 means we select the complete item.0 means we do not select the item.We cannot take a fraction of an item. Dynamic Programming (DP) is a method of solving a problem by breaking it into smaller subproblems and storing their results to avoid solving the same subproblem repeatedly.In 0/1 Knapsack, we use a 2D table to store the maximum profit for different items and capacities.For every item, we have two choices:

1. Include the item: Add its profit and reduce the remaining capacity.
2. Exclude the item: Keep the previous maximum profit.We choose whichever gives the maximum profit.

Complexity	**Value**	**Reason**
Time	O(n × W)	Two nested loops
Space	O(n × W)	2D DP table

Here:n = Number of items.W = Knapsack capacity. 1. Budget allocation

A company has a fixed budget and several projects. Each project has a cost and expected profit. The company selects projects to maximize profit without exceeding the budget.2. Cargo loading:A delivery vehicle has a limited weight capacity. Each package has a weight and value. The vehicle selects complete packages to maximize the total value without exceeding its capacity.

What is a DP table?It is a table that stores the maximum profit for different numbers of items and capacities.

The Greedy Method is an algorithmic approach that selects the best possible choice at each step to obtain an optimal solution.In Fractional Knapsack: time (ologn):sorting takes maxi time space o(n) We store n items in a list.

- Calculate the profit-to-weight ratio of every item.
- Sort items in descending order of the ratio.
- Select the item with the highest ratio first.
- If the complete item cannot fit, take only the required fraction.
- Continue until the knapsack is full.

Why Greedy here?Because selecting items with the highest profit per unit weight first gives the maximum profit for the fractional knapsack problem.